Operating System Practicals: Theory and Algorithms

Subject: Operating Systems (Ubuntu / Linux, C and Shell). Practicals 1 to 8.

Practical 1: Shell Programming (Address Book)

Theory

- A shell is a command interpreter. It takes commands from the user and gives them to the Linux kernel.
- A shell script is a text file that contains shell commands. It is run with `./script.sh`.
- The address book stores each record (Name | Phone | Email) as one line in a text file, `addressbook.txt`.
- Commands used: `echo` (print), `read` (take input), `case` (menu choice), `while` (loop), `cat` (show file), `grep -v` (remove a matching line), `sed` (replace text in a line), `>` (overwrite file), `>>` (append to file).

Algorithm

1. Start. Set the file name to `addressbook.txt`.
2. Repeat the steps below in a loop and show the menu: 1 Create, 2 View, 3 Insert, 4 Delete, 5 Modify, 6 Exit.
3. Read the choice.
4. If choice = 1: write the header line `NAME | PHONE | EMAIL` into the file (this creates a new empty book).
5. If choice = 2: display the file using `cat`.
6. If choice = 3: read name, phone and email, then append `name | phone | email` to the file.
7. If choice = 4: read the name, remove the line that begins with that name using `grep -v`, and save the file.
8. If choice = 5: read the old name and the new name, phone and email, then replace that line using `sed`.
9. If choice = 6: break out of the loop. Any other number: print Invalid choice.
10. Stop.

Practical 2: Process Management (fork, wait, zombie, orphan)

Theory

- A process is a program in execution. Every process has a PID (process id) and a PPID (parent's id).
- `fork()` creates a new process (the child) as a copy of the calling process (the parent). It returns 0 in the child, the child's PID in the parent, and -1 on failure.
- `wait()` makes the parent sleep until a child finishes. This also lets the parent collect the child's exit status.
- **Zombie process:** the child has finished but the parent has not yet called `wait()`. Its entry stays in the process table. `ps` shows state **Z** (defunct).
- **Orphan process:** the parent finishes before the child. The child is adopted by the `init` / `systemd` process (PID 1), so its PPID becomes 1.

Algorithm (sorting with fork and wait)

1. Start. Read n and then n integers into an array.
2. Call `fork()`.
3. If the return value is 0 (child): sort the array in descending order, print it, and exit.
4. If the return value is greater than 0 (parent): sort the array in ascending order and print it.
5. Parent calls `wait()` and waits for the child to finish.
6. Print Child finished. Stop.

Algorithm (Zombie)

1. Call `fork()`.
2. The child prints a message and exits immediately.
3. The parent sleeps for 15 seconds and does not call `wait()`.
4. In this time, run `ps -o pid,ppid,stat,cmd` in another terminal. The child shows state Z.

Algorithm (Orphan)

1. Call `fork()`.
2. The parent prints a message and exits immediately.
3. The child prints its PPID, sleeps 3 seconds, then prints its PPID again. The new PPID is 1.

Practical 3: Thread Synchronization (Producer-Consumer)

Theory

- A thread is a light-weight process. Threads of one process share the same memory.
- Producer-consumer (bounded buffer) problem: the producer puts items into a buffer of fixed size, and the consumer takes items out. The producer must wait if the buffer is full. The consumer must wait if it is empty. Both must not use the buffer at the same time.
- A **counting semaphore** is an integer variable with two operations: `wait()` (decrease, block if the value is 0) and `post()` (increase).
- A **mutex** gives mutual exclusion. Only one thread can be in the critical section at a time.
- Semaphores used: `empty` = N (number of free slots), `full` = 0 (number of filled slots), and one `mutex`.
- Compile with `-pthread`.

Algorithm

1. Start. Set empty = N, full = 0, mutex = unlocked, in = 0, out = 0.
2. Create the producer thread and the consumer thread.
3. Producer, repeated for each item:
   a. `wait(empty)`.
   b. `lock(mutex)`.
   c. Put the item in `buffer[in]` and set `in = (in + 1) % N`.
   d. `unlock(mutex)`.
   e. `post(full)`.
4. Consumer, repeated for each item:
   a. `wait(full)`.
   b. `lock(mutex)`.
   c. Take the item from `buffer[out]` and set `out = (out + 1) % N`.
   d. `unlock(mutex)`.
   e. `post(empty)`.
5. The main thread waits for both threads with `pthread_join`. Stop.

Practical 4: Dining Philosophers (Deadlock-free)

Theory

- Five philosophers sit around a table. There is one fork between each pair, so there are 5 forks. A philosopher needs both the left and the right fork to eat.

- **Deadlock:** if all five pick up their left fork at the same time, each waits for the right fork forever. Nobody can eat.
- **Starvation:** a philosopher may wait for a very long time and never get to eat.
- Solution used: a semaphore `room` with the value N-1 = 4. At most 4 philosophers try to pick up forks at the same time, so at least one philosopher can always get both forks. This avoids deadlock (it breaks circular wait).
- Each fork is a semaphore with the value 1.

Algorithm

1. Start. Set `fork[0..4] = 1` and `room = 4`.
2. Create 5 philosopher threads (id 0 to 4).
3. Each philosopher repeats:
   a. Think.
   b. `wait(room)`.
   c. `wait(fork[i])` (left fork).
   d. `wait(fork[(i+1) % 5])` (right fork).
   e. Eat.
   f. `post(fork[(i+1) % 5])`.
   g. `post(fork[i])`.
   h. `post(room)`.
4. The main thread joins all threads. Stop.

Practical 5: Deadlock Avoidance (Banker's Algorithm)

Theory

- Deadlock avoidance means the OS grants a resource only if the system stays in a **safe state**.
- A **safe state** is a state where there is at least one order (safe sequence) in which all processes can finish.
- Data structures (n processes, m resource types):
  - **Available[m]**: free resources of each type.
  - **Max[n][m]**: the most each process may need.
  - **Allocation[n][m]**: what each process holds now.
  - **Need[n][m] = Max - Allocation**: what each process may still ask for.
- The safety algorithm uses `Work = Available` and `Finish[i] = false` for all i.

Algorithm (Safety Algorithm)

1. Start. Read n, m, Allocation, Max and Available.
2. Calculate Need = Max - Allocation.
3. Set Work = Available and Finish[i] = false for all processes.
4. Find a process i with Finish[i] = false and Need[i] <= Work.
5. If found: set Work = Work + Allocation[i], set Finish[i] = true, add Pi to the safe sequence, and go back to step 4.
6. If not found: go to step 7.
7. If Finish[i] = true for all i, the system is in a SAFE state and the safe sequence is printed. Otherwise, the system is UNSAFE.
8. Stop.

Example used: 5 processes (P0 to P4), 3 resources, Available = 3 3 2. Result: SAFE, sequence P1 -> P3 -> P4 -> P0 -> P2.

Practical 6: Interprocess Communication using FIFOs

Theory

- IPC lets two processes exchange data.
- A **FIFO (named pipe)** is a special file in the file system, created with `mkfifo`. Data is read in first-in first-out order. Because it has a name, two **unrelated** processes can use it.
- One FIFO carries data in one direction only. For **full duplex** (two-way) communication we use two FIFOs: `pipe1` (Process 1 to Process 2) and `pipe2` (Process 2 to Process 1).
- System calls used: `mkfifo`, `open`, `read`, `write`, `close`.
- Opening a FIFO blocks until the other process opens its other end.

Algorithm

1. Create the two FIFOs: `mkfifo pipe1 pipe2`.
2. **Process 1:**
   a. Read a sentence from the keyboard. If it is `exit`, stop.
   b. Open `pipe1` for writing, write the sentence, and close it.
   c. Open `pipe2` for reading, read the result, and close it.
   d. Display the result on the screen. Go back to step 1.
3. **Process 2** (repeats forever):
   a. Open `pipe1` for reading and read the sentence.
   b. Count characters, words (groups of non-space characters) and lines (number of `\n`).

c. Write the three counts into `result.txt`.

d. Read the contents of `result.txt`.

e. Open `pipe2` for writing and write the contents into it.

Practical 7: Interprocess Communication using Pipes

Theory

- A **pipe** is an unnamed one-way channel in memory, created with `pipe(fd)`. `fd[0]` is the read end and `fd[1]` is the write end.
- It works only between **related** processes (parent and child), because the child inherits the file descriptors through `fork()`.
- For **full duplex** communication we use two pipes: `p1` (parent to child) and `p2` (child to parent).
- Each process should close the pipe ends that it does not use.
- `read()` returns 0 (end of file) when all write ends are closed.

Algorithm

1. Start. Create two pipes: `pipe(p1)` and `pipe(p2)`.
2. Call `fork()`.
3. **Parent:**
   a. Read the file path from the user.
   b. Write the path into `p1`, then close the write end of `p1`.
   c. Close the write end of `p2`.
   d. Read from `p2` until end of file and display the data on the screen.
   e. Call `wait()` for the child.
4. **Child:**
   a. Read the path from `p1`.
   b. Open the file. If it cannot be opened, write the message File not found into `p2`.
   c. Otherwise, read the file and write its contents into `p2`.
   d. Exit.
5. Stop.

Practical 8: Disk Scheduling (SSTF, SCAN, C-LOOK)

Theory

- The disk scheduling algorithm decides the order in which pending disk requests are served. The goal is to **reduce total head movement (seek time)**.
- Total head movement is the sum of |current position - next position|.
- Here the head starts by **moving away from the spindle**, which means towards the **higher** cylinder numbers.
- **SSTF (Shortest Seek Time First):** always serve the request nearest to the current head position. It can cause starvation of far-away requests.
- **SCAN (elevator):** the head moves in one direction up to the last cylinder of the disk (here cylinder 199), serving requests on the way. Then it reverses and serves the remaining requests.
- **C-LOOK:** the head moves only up to the last (highest) request, not to the end of the disk. Then it jumps to the lowest request and moves upward again, serving the rest in the same direction.

Algorithm (SSTF)

1. Read the requests and the head position. Mark all requests as not served.
2. Find the unserved request with the smallest distance from the head.
3. Add that distance to the total, move the head there, and mark it as served.
4. Repeat steps 2 and 3 until all requests are served. Print the order and the total.

Algorithm (SCAN)

1. Sort the requests in ascending order.
2. Move the head upward and serve all requests greater than or equal to the head. Add each distance to the total.
3. Go to the last cylinder (size - 1) and add that distance.
4. Reverse the direction. Serve the remaining requests (smaller than the head position) in descending order, and add the distances.
5. Print the order and the total.

Algorithm (C-LOOK)

1. Sort the requests in ascending order.
2. Move the head upward and serve all requests greater than or equal to the head.
3. Jump from the highest request to the lowest request and add that distance.
4. Serve the remaining requests in ascending order (the ones smaller than the initial head position).
5. Print the order and the total.

Example used: 200 cylinders (0 to 199), requests 98 183 37 122 14 124 65 67, head = 53.

- SSTF: 53, 65, 67, 37, 14, 98, 122, 124, 183. Total = **236**.

- SCAN: 53, 65, 67, 98, 122, 124, 183, 199, 37, 14. Total = **331**.
- C-LOOK: 53, 65, 67, 98, 122, 124, 183, 14, 37. Total = **322**.